

目录

第一章 规格总览与核心结论	2
第二章 数据来源与提取方法	2
2.1 数据来源	2
2.2 提取规则	2
第三章 能力类型系统	3
第四章 快速通道编号表	4
4.1 ctrlmem 页表快速通道	4
4.2 SMMU 快速通道	5
第五章 Linux 兼容系统调用层	6
第六章 引导交接与布局断言	7
第七章 声明面统计与开源边界	7
第八章 重实现路线评估	8
附录 清单文件使用说明	8

第一章 规格总览与核心结论

本规格书从 HarmonyOS 官方开源合规包（SCA-AL00，HarmonyOS 6.1.0）中的微内核项目 hm-verif-kernel 公开头文件出发，对其应用二进制接口（ABI）做机器化提取与归档。提取对象覆盖内核 UAPI 头（kernel/include/mapi/uapi）与架构侧 UAPI 头（kernel/arch/aarch64/include/mapi/uapi），目标是回答一个问题：闭源的微内核本体，其对外契约到底被公开部分钉死了多少，以及按这些契约重实现一个 ABI 兼容内核的可行性与工作量。

197

UAPI 头文件

21

能力对象类型

819

编号操作（76 张表）

705

Linux 兼容系统调用

核心结论有四条。第一，该内核的能力对象类型系统被 CAPTYPE 宏体系完整公开：21 个类型、每个类型的数字编号、授权粒度（细粒度/粗粒度/禁授权）、方法表与类型化错误码全部可还原，这相当于把整个内核的对象模型交了出来。第二，快速通道（fastcall）操作以编号常量形式公开，ctrlmem 页表快速通道 14 个编号操作、SMMU 快速通道 21 个操作构成最小可执行回路的全部入口。第三，Linux 兼容层直接复用 Linux 系统调用编号（313 个主编号加 392 个兼容编号），并以编译期断言锁定，用户态二进制的系统调用兼容性是被保证的。第四，头文件钉死的是"接口与 ABI"；调度策略、页分配算法、异常向量等"语义与实现"不在其中，这些构成重实现的全部工作量。

需要特别说明的是能力类型编号中 16 号的缺席：21 个公开类型的编号为 1 至 15 与 17 至 22，16 号没有任何公开头文件定义。这要么是保留编号，要么对应某个完全未公开的内核对象类型，它本身即是内核结构的旁证之一。

第二章 数据来源与提取方法

2.1 数据来源

全部数据来自官方开源合规包内的两个目录树：其一为 kernel/include/mapi，包含 UAPI 子树（197 个头文件，面向用户态的稳定接口）与内核侧 mapi/hmkernel 子树；其二为 kernel/arch/aarch64/include/mapi，包含 ARM64 架构相关的 UAPI 头，其中 hmasm/boot/sysproc_bootargs.h 定义了引导交接布局，hmasm/lscno_def.h 定义了 Linux 兼容系统调用编号。为便于复核，原始路径在清单 JSON 中逐条保留。

2.2 提取规则

提取器（extract_abi.py，随本规格书一同交付）按五类规则工作：CAPTYPE 宏解析能力类型及其授权与方法表；编号宏扫描捕获 #define __前缀_操作 数字 形式的快速通道编号；枚举扫描捕获 enum __xxx_cmd 形式的操作表（如 SMMU）；LSCNO_DEF 与 LSCNO_COMPAT_DEF 捕获 Linux 兼容系统调用编号；断言宏扫描统计结构布局的编译期校验。方法的注释级分类（capcall 专有/兼备 fastcall）保留在原始头文件中，提取器不猜测语义。

已知局限有三：条件编译分支未逐一展开，少数表格的取值以宏表达式给出而未捕获；CAPERRNO 之外的内联注释不进入清单；架构侧头文件仅处理 AArch64。这些局限影响覆盖率的绝对数值，不影响接口存在性判定。

第三章 能力类型系统

能力（capability）是该内核一切特权操作的通行证：线程、地址空间、中断控制器、调度器都是能力对象，持有能力方可调用对应方法。CAPTYPE 宏为每个类型给出三件事：数字编号（出现在内核对象头与授权检查中）、授权策略（CAPFINEGRANT 细粒度、CAPCOARSEGRANT 粗粒度、CAPNOGRANT 禁止授权，以及 GRANTMOVE 变体）、与方法表（每个方法对应一个 capcall/fastcall 编号）。下表为 21 个类型的完整清单。

编号	类型	授权粒度	授权移动	方法数	错误码
1	CNode	COARSE	NO	44	41
2	Thread	COARSE	NO	45	3
3	PMEM	ALL	NO	1	5
4	Channel	ALL	NO	2	0
5	Activation	NO	NO	11	10
6	ActivationPool	COARSE	COARSE	25	19
7	VSpace	ALL	NO	12	11
8	SysCtrl	ALL	NO	67	0
9	IrqCtrl	FINE	ALL	13	1
10	CpuContext	NO	NO	3	0
11	Scheduler	ALL	NO	1	0
12	VM	FINE	NO	21	0
13	Timer	ALL	NO	4	1
14	Module	FINE	NO	4	24
15	Permission	NO	NO	1	2
17	Livepatch	ALL	NO	4	0
18	EcEp	NO	NO	3	0
19	EcChn	NO	NO	0	0
20	DsuContext	NO	NO	3	0
21	FDTable	ALL	NO	29	0
22	Epoll	ALL	NO	8	0

表 3-1 能力对象类型全表（编号 16 缺席）

从规模看，SysCtrl（67 个方法）是系统控制中枢，覆盖电源、时钟、复位等平台级操作；CNode（44 个方法）是能力容器本身，负责能力的复制、吊销与传递；Thread（45 个方法）支持唤醒、挂起、绑定 CPU、名字设置等完整线程控制面。从授权策略看，Thread 与 VM 选择粗粒度授权并禁止授权移动，而 IrqCtrl 选择细粒度授权且允许全部授权移动并禁用默认查找——中断能力可以在进程间精确移交，这与用户态驱动框架（devhost/UDK）持有中断能力的设计互为印证。

以 Thread 类型为例，其配置字以 32 位高位掩码表达创建语义：调用者地址空间共享（0x1）、创建即挂起（0x2）、克隆用户上下文（0x4）、vfork（0x40）、内核辅助线程（0x1000）、fork 时复位（0x2000）等。这些位域与 Linux 的 clone 标志存在清晰的映射关系，是 sysmgr 实现 fork/clone 语义的基石。

第四章 快速通道编号表

除能力方法外，内核以编号常量暴露快速通道操作。全库共提取 76 张表、819 条编号，按规模排序的主要表格如下。其中 CTRLMEM 一张表即含 176 条，原因是它同时编码了命令号与位域掩码（如页表项访问位、只读位的置位/清除掩码），把"命令+参数编码"全部宏化。

编号表	条数	所在头文件（首个）
CTRLMEM	176	ctrlmem/copgstrtbl.h
SCHED	103	power/sched_indicator.h
ACTV	64	capability/captype_activation.h
PGSTR	60	pgstr/pgstr.h
VREGION	35	ctrlmem/kobj/vregion.h
BOOTTRACE	34	boottrace.h
FUTEX	29	futex.h
SMMU	22	smmu_syscall.h
KOBJVS	17	ctrlmem/kobj/vspace/rangeop.h
SYSPG	16	ctrlmem/syspg/syspg.h

表 4-1 编号表规模排行（前十）

4.1 ctrlmem 页表快速通道

ctrlmem 是内核与 sysmgr 之间的控制内存机制，其系统调用子表定义了 14 个编号操作，构成地址空间管理的快速路径：映射用户页（MAP_PGVA）、批量映射（BATCH_MAP_PGVA）、TLB 聚合卸载（TLBGATHER_UNMAP）、区间保护修改（RANGE_MPROTECT）、区间卸载（RANGE_MUNMAP）、页串映射（MAP_PGSTRVA）等。这组编号就是 sysmgr 代替内核管理页表时的全部入口。

编号	操作
1	__CTRLMEM_SYSCALL_COPGSTR_TBL_VAL_OP_REGISTER

编号	操作
2	__CTRLMEM_SYSCALL_COPGSTRTBL_VAL_OP_UNREGISTER
1	__CTRLMEM_SYSCALL_PGTBLKLL_VAL_OP_COPGSTR
0	__CTRLMEM_SYSCALL_UNKNOWN
1	__CTRLMEM_SYSCALL_MAP_PGVA
2	__CTRLMEM_SYSCALL_CLEAR_ABIT
3	__CTRLMEM_SYSCALL_CLEAN_SET_KOBJVS
4	__CTRLMEM_SYSCALL_REMAP_PGVA_KOBJVS
5	__CTRLMEM_SYSCALL_MAP_PGSTRVA
6	__CTRLMEM_SYSCALL_BATCH_MAP_PGVA
7	__CTRLMEM_SYSCALL_TLBGATHER_UNMAP
8	__CTRLMEM_SYSCALL_BATCH_TLBG_UNMAP
9	__CTRLMEM_SYSCALL_RANGE_MPROTECT
10	__CTRLMEM_SYSCALL_RANGE_MUNMAP
11	__CTRLMEM_SYSCALL_VSMKDEAD
12	__CTRLMEM_SYSCALL_CHANNELPG_UNLOCK
13	__CTRLMEM_SYSCALL_PGSTR_NML2IOFA
14	__CTRLMEM_SYSCALL_MAX
0	__CTRLMEM_SYSCALL_TLBGATHER_OP_UNMAP_PARTIAL
1	__CTRLMEM_SYSCALL_TLBGATHER_OP_FLUSH

表 4-2 ctrlmem 快速通道系统调用（14 个编号操作）

4.2 SMMU 快速通道

SMMU（IOMMU）命令表以枚举形式公开，21 个操作覆盖设备attach、映射与批量映射、IOVA 反查物理地址、TLB 按上下文失效、TTW 刷新、TCU 电源管理、PMU 配置等。用户态驱动申请映射、内核独占页表的权力结构，在编号层面一目了然。

值	命令	语义
0	__SMMU_SMMU_MAP	
1	__SMMU_SMMU_UNMAP	
2	__SMMU_SMMU_MAP_SG	
3	__SMMU_SMMU_MAP_BATCH	
4	__SMMU_SMMU_UNMAP_BATCH	
5	__SMMU_SMMU_IOVA_TO_PHYS	
6	__SMMU_SMMU_ATTACH_DEV	

值	命令	语义
7	__SMMU_SMMU_ATTACH_NEW_DEV	
8	__SMMU_SMMU_SVM_DETACH_DEV	
9	__SMMU_SMMU_SMMU_DEVICE_REMOVE	
10	__SMMU_SMMU_SMMU_DOMAIN_FREE	
11	__SMMU_SMMU_TLB_INV_CONTEXT	
12	__SMMU_SMMU_TLB_INV_CONTEXT_DEV	
13	__SMMU_SMMU_FLUSH_TTWC	
14	__SMMU_SMMU_FLUSH_PGTBL_CACHE	
15	__SMMU_SMMU_TCU_POWER_ON	
16	__SMMU_SMMU_TCU_POWER_OFF	
17	__SMMU_SMMU_GET_TTBR	
18	__SMMU_SMMU_GET_TCU_INFO	
19	__SMMU_SMMU_PMU_TCU_CONFIG	
20	__SMMU_SMMU_UNMAP_INV_CACHE	
21	__SMMU_SMMU_CMD_MAX	

表 4-3 SMMU 快速通道命令表（21 个操作）

第五章 Linux 兼容系统调用层

lscno_def.h 以 LSCNO_DEF(名称, 编号) 的形式定义了 313 个本地系统调用, 另有 392 个兼容编号 (LSCNO_COMPAT_DEF)。机制上, 内核侧以枚举生成 __NR_ 与 __SYS_ 双前缀常量, 用户态则用 raw_static_assert 断言与 glibc 的 __NR_ 编号逐一相等——也就是说 liblinux 载入的 Linux 程序与驱动发起系统调用时, 编号无需翻译直接进入兼容层, 兼容性由编译期锁定而非运行期约定。

编号表内容从 io_setup (0 号) 起按 Linux aarch64 通用编号排列, 覆盖文件、内存、进程、信号、网络全套。这一层与 devhost 的 liblinux 后端互为表里: 前者是系统调用的"号", 后者是系统调用的"实现"。

项目	数值	说明
主编号	313	LSCNO_DEF, 与 Linux __NR_ 对齐
兼容编号	392	LSCNO_COMPAT_DEF, 32 位兼容路径
编号起点	0	io_setup
锁定方式	raw_static_assert	用户侧编译期断言一致

表 5-1 lsyscall 层概览

第六章 引导交接与布局断言

sysproc_bootargs.h 定义了微内核移交给首个系统进程（sysmgr）的 72 字节引导参数结构，并以编译期断言锁定每个字段的偏移。这份"禅让诏书"是内核与系统服务器之间的第一契约：内核完成早期汇编与初始页表构建后，将根页表、用户程序区间、安全策略库虚拟地址一并移交。

字段	偏移(字节)	语义
load_start	0	映像加载起始地址
canary	8	栈保护金丝雀值
superpolicydb_va_st	32	SELinux 超级策略库虚拟地址（可选）
pgd	40	根页表基地址
uapps_start	48	用户程序区间起点
uapps_end	56	用户程序区间终点
mmap_rnd	64	mmap 随机化熵

表 6-1 sysproc 引导参数布局（结构总长 72 字节，10 处断言）

结构开头两个指令字保留为 nop 与跳转指令，供内核跳入 sysmgr 的 head.S；配置了策略库字段时（CONFIG_POLICYDB_USE_SUPERVS），SELinux 策略数据库的虚拟地址在第 32 字节处交付。这一设计把"安全策略随内核引导"写进了 ABI。

第七章 声明面统计与开源边界

按子系统统计 UAPI 头中的函数声明共 2,361 处。追踪类（trace/events）占大头，这反映内核的可观测性设计：跟踪点作为稳定 ABI 交付。核心声明集中在页表处理、控制内存与能力目录，与第三、四章的编号表互为印证。

子系统	声明数
trace	1408
.	329
events	171
pgtblhdlr	152
ctrlmem	96
capability/.	81
track_info	60
attributes	17

表 7-1 函数声明按子系统分布（前八）

结合本系列分析的整体结论，开源边界如下表所示。头文件交付的是完整 ABI：对象模型（第三章）、快速通道（第四章）、系统调用编号（第五章）、引导契约（第六章）。闭源的微内核与 sysmgr 二进制，恰好处在这份规格书的每一行约束之下。

组件	状态	本规格书覆盖
微内核本体（调度/内存/能力运行时）	闭源	ABI 全覆盖
sysmgr 系统服务器	闭源	intapi 契约覆盖
UAPI 头（465 个）	开源	本文数据源
用户态世界（devhost/UDK/liblinux）	开源	消费方佐证

表 7-2 开源边界与本规格书的覆盖关系

第八章 重实现路线评估

以 ABI 兼容为目标重实现该内核，可行性取决于三点：契约完备度、调用方可观测性、工作量量级。契约完备度方面，本规格书显示对象模型与快速通道均已钉死；调用方可观测性方面，devhost/UDK/liblinux 全部开源，构成最苛刻的测试用例来源；工作量方面，缺口的全部在实现语义：调度策略、物理页分配、TLB 与缓存维护、异常向量与上下文切换、GICv3/SMMUv3 内核侧驱动、能力运行时的授权与吊销逻辑。

里程碑	内容	依赖规格
M1 最小回路	启动、CNode/Thread 能力、ctrlmem 映射	第三、四、六章
M2 系统进程	bootargs 移交、sysmgr 存根、页表服务	第六章
M3 兼容层	lsyscall 透传 + libdh-linux 载入	第五章
M4 驱动栈	devhost + SMMU/irq 能力交付	第四章

表 8-1 重实现里程碑路线（研究性重建口径）

合规边界需要明确：重实现只能以公开头文件与开源调用方为规格来源，不得反汇编闭源二进制，不得使用任何泄露实现（本系列已证实泄露包亦无内核实现），产物不得使用 HarmonyOS 或鸿蒙商标。此路线与 Wine、ReactOS 等先例的合法实践一致。工作量量级按 seL4 参照折算，研究性重建的最小回路为个人年级别，完整 ABI 兼容替换为三至五人年级别。

附录 清单文件使用说明

随本规格书交付机器可读清单 abi_inventory.json（约 147KB）与提取器 extract_abi.py。清单顶层键为：meta（统计元数据）、capability_types（能力类型数组，含 cap_id、name、grant、grant_move、methods、errno、flags）、numbered_ops（编号表字典，键为表前缀，值为 op/value/header 列表）、lsyscalls 与 lsyscalls_compat（Linux 兼容编号数组）、bootargs（引导参数字段与偏移）、layout_asserts（布局断言分布）、declarations（声明按子系统统计）。提取器用法：python3 extract_abi.py，可在新版本源码包上直接重跑以做版本间 ABI 对比。